

Alto Money: v1 Basemarket Changes

Security Review

Cantina Solo review by:
Phaze, Lead Security Researcher

July 1, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Gas Optimization	4
3.1.1	Liquidation token flow could be simplified or separated more clearly	4
3.1.2	Advanced fee strategy can pack fee values into smaller storage types	5
3.2	Informational	5
3.2.1	In-place upgrades from legacy USM to advanced USM can misconfigure access state	5
3.2.2	Liquidation events could use the original liquidator for easier indexing	7
3.2.3	Supported market removal could avoid external validation calls	7
3.2.4	Independent USM caps can create liquidation configuration footguns	8
3.2.5	USM seize could handle accrued stable-token fees more explicitly	9
3.2.6	Advanced USM compatibility is not explicitly discoverable	9
3.2.7	DUSD is closely linked to frxUSD through the liquidation USM	10

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A security review is a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While the review endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that a security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Alto is a decentralized credit protocol built around DUSD, focused keeping risk contained by isolating markets, using a single stablecoin and partial liquidations.

From Jun 22nd to Jun 25th the security researchers conducted a review of [v1](#) on commit hash [da40ba42](#). A total of **9** issues were identified:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	2	1	1
Informational	7	3	4
Total	9	4	5

2.1 Scope

The security review had the following components in scope for [v1](#) on commit hash [da40ba42](#):

```
contracts
├── lending
│   ├── AltoBaseMarket.sol
│   └── liquidation
│       └── LiquidationPeriphery.sol
└── usm
    ├── AdvancedPermissionsFeeStrategy.sol
    ├── AdvancedPermissionsUsm.sol
    └── DUSDAdvancedPermissionsUsm.sol
```

3 Findings

3.1 Gas Optimization

3.1.1 Liquidation token flow could be simplified or separated more clearly

Severity: Gas Optimization.

Context: [AltoBaseMarket.sol#L624](#).

Description: The liquidation flow currently mixes two design approaches. In one direction, `AltoBaseMarket` is aware of authorized liquidation periphery contracts: when `msg.sender` is an authorized periphery, the market decodes `originalLiquidator` from `liquidationData` and passes that address to the liquidation engine. This lets the core market account for the economic liquidator when checking liquidation permissions.

In the token flow, however, the market still sends seized collateral to `msg.sender`, which is the periphery during periphery liquidations. The periphery then forwards that collateral to the original liquidator. This creates an additional token hop:

```
market → liquidation periphery → original liquidator
```

Since the market already has special handling for authorized periphery callers and already decodes `originalLiquidator`, it could theoretically use that additional context to send collateral directly to the original liquidator:

```
market → original liquidator
```

The current approach is not a security issue by itself. It is a design tradeoff. Sending tokens to `msg.sender` is a common convention that keeps core market contracts agnostic to periphery-specific flows. However, the market is no longer fully agnostic because it already has periphery-specific authorization and decoding logic. This creates an intermediate design where the market knows enough about the periphery to support priority checks, but does not use that knowledge to simplify token routing or event indexing.

An alternative architecture would be to keep `AltoBaseMarket` fully sender-agnostic and move the concept of periphery-specific original liquidators into the liquidation engine or periphery layer. Conversely, if the market intentionally remains aware of authorized periphery contracts, it could use that awareness consistently for conveniences such as direct collateral routing and simpler indexing semantics.

Recommendation: Consider choosing one convention more explicitly.

One option is to keep the current core-market convention: the market always transfers collateral to `msg.sender`, and periphery contracts are responsible for any forwarding. This keeps token movement simple from the market's perspective, but accepts the extra token hop and event-matching burden.

Another option is to lean into the existing periphery-aware market design. In that model, when `msg.sender` is an authorized liquidation periphery, the market could send seized collateral directly to `originalLiquidator` while still invoking the periphery callback for repayment orchestration. This would avoid the intermediate collateral transfer and make the market's periphery-specific behavior more internally consistent.

If direct collateral transfer is adopted, ensure the callback interface and repayment assumptions remain clear: the periphery would no longer custody the seized collateral before calling the liquidator, so any periphery callback data should assume the original liquidator has received the collateral directly.

Alto: Acknowledged.

Phaze: Acknowledged.

3.1.2 Advanced fee strategy can pack fee values into smaller storage types

Severity: Gas Optimization.

Context: [AdvancedPermissionsUsmFeeStrategy.sol#L23](#).

Description: `AdvancedPermissionsFeeStrategy` stores buy and sell fee values as `uint256`, including default fees and overridden fee data. However, all configured fee values are validated to be below `MAXIMUM_FEE_PERCENT`, which is `5000` basis points.

This means the fee values fit comfortably in a much smaller integer type such as `uint16`. The current `OverriddenFeeData` layout also stores a boolean override flag next to a full-width fee value, causing each mapping entry to use more storage than necessary. This is not a security issue, but it increases storage writes for fee configuration and makes the fee data layout larger than the value range requires.

Recommendation: Consider using compact internal storage for fee values. If preserving the external interface is preferred, the public ABI can continue accepting and returning `uint256` while the implementation stores the values in a packed internal struct.

```
struct PackedFeeData {
    bool isOverridden;
    uint16 fee;
}

uint16 internal _buyFee;
uint16 internal _sellFee;

mapping(address => PackedFeeData) internal _overrideBuyFee;
mapping(address => PackedFeeData) internal _overrideSellFee;
mapping(bytes32 => PackedFeeData) internal _buyFeeByRoleHash;
mapping(bytes32 => PackedFeeData) internal _sellFeeByRoleHash;
```

The setter functions can continue validating `fee < MAXIMUM_FEE_PERCENT` before casting to `uint16`, and the getter functions can cast back to `uint256` at the ABI boundary.

Alto: Fixed in [PR 535](#).

Phaze: Fix verified.

3.2 Informational

3.2.1 In-place upgrades from legacy USM to advanced USM can misconfigure access state

Severity: Informational.

Context: [DUSDAAdvancedPermissionsUsm.sol#L16](#).

Summary: Upgrading an existing `DUSDUsm` proxy directly to `DUSDAAdvancedPermissionsUsm` would reinterpret the stored access-control byte under different enum semantics. This can produce unexpected buy or sell access settings after the upgrade.

The team indicated that legacy USM deployments are not intended to be upgraded in place to the advanced implementation. The advanced USMs are expected to be deployed as new contracts, so this is an informational deployment and migration hardening note.

Description: The legacy `Usm` contract stores a single `_accessMode` enum at storage slot 4 offset 16 to gate both swap directions. The advanced implementation keeps the same slot and offset for `_buyDirectionAccess`, then introduces `_sellDirectionAccess` at slot 4 offset 17.

The enum values are not equivalent between the two implementations:

```
enum AccessMode {
    PERMISSIONLESS, // 0
    PERMISSIONED    // 1
}

enum DirectionAccess {
    PERMISSIONED, // 0
    PERMISSIONLESS // 1
}
```

As a result, a direct implementation upgrade would not preserve access semantics. A legacy permissioned USM stores `_accessMode==1`, which the advanced implementation would read as `_buyDirectionAccess==PERMISSIONLESS`. The newly added `_sellDirectionAccess` would read from previously unused padding and would typically decode to `PERMISSIONED`.

Conversely, a legacy permissionless USM stores `_accessMode==0`, which the advanced implementation would read as `_buyDirectionAccess==PERMISSIONED`, while `_sellDirectionAccess` would also typically decode to `PERMISSIONED`. Since legacy role holders have `SWAPPER_ROLE` rather than `BUY_SWAPPER_ROLE` or `SELL_SWAPPER_ROLE`, the upgraded contract may also lack the role grants expected by the advanced implementation.

The advanced initializer cannot repair this state on an already initialized proxy because `initialize()` uses the standard `initializer` modifier. The advanced implementation also does not expose a dedicated migration reinitializer that maps the legacy access mode into explicit buy and sell direction settings.

Impact Explanation: If a legacy proxy were upgraded directly, the resulting access configuration could be more permissive or more restrictive than intended. In the permissioned-to-advanced case, buy-side access may become permissionless; in the permissionless-to-advanced case, ordinary swaps may become unavailable until the new roles and access settings are configured.

Likelihood Explanation: This requires an authorized operator to intentionally perform an in-place upgrade from `DUSDUsm` to `DUSDAdvancedPermissionsUsm`. The team has indicated that this is not the intended migration path and that advanced USMs will be deployed as new contracts, which makes the practical likelihood low.

Recommendation: Consider documenting that legacy `DUSDUsm` proxies must not be upgraded directly to `DUSDAdvancedPermissionsUsm`. If an in-place migration is ever needed, add an explicit migration path that preserves storage compatibility, maps the legacy `_accessMode` into both direction-access fields, and grants the corresponding `BUY_SWAPPER_ROLE` and `SELL_SWAPPER_ROLE` permissions in the same governance action.

One approach could be to add a dedicated reinitializer on a storage-compatible migration implementation:

```
function initializeV2(DirectionAccess buyAccess, DirectionAccess sellAccess)
    external
    reinitializer(2)
    onlyRole(DEFAULT_ADMIN_ROLE)
{
    _updateAccessConfig(buyAccess, sellAccess);
}
```

For the currently planned approach, fresh advanced USM deployments avoid this storage reinterpretation issue.

Alto: Acknowledged.

Phaze: Acknowledged.

3.2.2 Liquidation events could use the original liquidator for easier indexing

Severity: Informational.

Context: [AltoBaseMarket.sol#L635](#).

Description: When liquidations are executed through `LiquidationPeriphery`, the base market receives the periphery contract as `msg.sender`. The market decodes the original liquidator from `liquidationData` and uses that value when preparing the liquidation, including for priority-liquidation checks.

The base market `Liquidation` event currently emits `msg.sender` as the `liquidator` field. For periphery liquidations, this means the base market event records the periphery contract rather than the actual liquidator. The periphery emits a separate `Liquidation` event that includes the original liquidator, so indexers need to match events emitted by two different contracts to reconstruct the full liquidation context.

This makes liquidation indexing and downstream analytics more cumbersome. Event matching across contracts is possible, but it is more tedious for node-backed indexers than reading the original liquidator directly from the base market event.

Recommendation: Consider changing the convention for the existing `liquidator` argument in the base market `Liquidation` event so that it represents the economic liquidator rather than always representing `msg.sender`. For direct liquidations, this remains equivalent to `msg.sender`; for periphery liquidations, it would use the decoded `originalLiquidator`.

One approach could be:

```
emit Liquidation(  
    borrower,  
-   msg.sender,  
+   originalLiquidator,  
    result.seizedCollateralAssets,  
    result.protocolSeizedCollateralFee,  
    result.repaidBorrowAmount,  
    result.repaidBorrowShares,  
    badDebt.assets,  
    badDebt.shares  
);
```

This preserves the event shape and avoids changing the event signature, while making periphery liquidations easier to index from the base market event alone.

Alto: Fixed in [PR 537](#).

Phaze: Fix verified.

3.2.3 Supported market removal could avoid external validation calls

Severity: Informational.

Context: [LiquidationPeriphery.sol#L144](#).

Description: `LiquidationPeriphery.setSupportedMarket()` validates that a market's `borrowToken()` matches the periphery `BORROW_TOKEN` before updating support status. This validation is useful when adding a new supported market because it prevents configuring a market for the wrong borrow token.

The same validation also runs when removing support. In normal operation this is harmless, but it means that removing a market still depends on an external call to the market succeeding and returning the expected token. If a supported market were paused in an unexpected way, upgraded incorrectly, or otherwise unable to respond to `borrowToken()`, the owner could be blocked from removing it from `supportedMarkets`.

This is not a security issue in normal conditions. It is a defensive administration and failure-isolation improvement: disabling support should ideally be possible even when the target market is in a degraded state.

Recommendation: Consider applying the borrow-token validation only when support is being enabled. When support is being disabled, allow the owner to clear the mapping without making an external call to the market.

One approach could be:

```
function setSupportedMarket(address market, bool isSupported) external onlyOwner {
    if (market == address(0)) revert ZeroAddress();
-   if (IMarket(market).borrowToken() != BORROW_TOKEN) revert InvalidInput();
+   if (isSupported && IMarket(market).borrowToken() != BORROW_TOKEN) revert
    ↪ InvalidInput();
    supportedMarkets[market] = isSupported;
    emit SupportedMarketSet(market, isSupported);
}
```

This preserves the safety check for onboarding new markets while making removal more robust during incident response or misconfiguration cleanup.

Alto: Fixed in [PR 533](#).

Phaze: Fix verified.

3.2.4 Independent USM caps can create liquidation configuration footguns

Severity: Informational.

Context: [AdvancedPermissionsUsm.sol#L468](#).

Description: The USM path used by liquidation periphery depends on two separate capacity controls. The USM must be able to accept the underlying asset being sold into it, which is limited by the USM exposure cap. The USM must also be able to mint the corresponding DUSD, which is limited by the DUSD minter ceiling for the USM address.

During deployment, the mint ceiling can be derived from the configured exposure cap. However, after deployment these are independent administrative controls:

```
USM exposure cap: AdvancedPermissionsUsm.updateExposureCap()
DUSD mint ceiling: DUSD.setMinterCeiling()
```

If the two values are not kept in sync, the lower effective cap becomes the binding limit for the liquidation route. For example, if the DUSD minter ceiling is raised but the USM exposure cap is not, a large liquidation can still revert when `sellAsset()` checks that the resulting exposure does not exceed `_exposureCap`. Conversely, if the exposure cap is raised but the DUSD minter ceiling is not, the USM can still be unable to mint the DUSD required for repayment.

This is most relevant because the liquidation periphery is intended to provide liquidators with reliable access to DUSD even when ordinary market liquidity is insufficient. If either cap is stale or misconfigured, liquidation cascades may fail despite the other cap being set high enough. The team indicated that both values are intended to be set to equal, arbitrarily high values for the liquidation USM; the concern is mainly that maintaining two independent caps creates an operational footgun.

Recommendation: Consider reducing the chance of configuration drift between the USM exposure cap and the DUSD minter ceiling by consolidating the capacity control.

One approach could be to rely on the DUSD minter ceiling as the canonical cap and remove, disable, or bypass the advanced USM exposure cap for liquidation-oriented USMs. The DUSD contract already limits how much DUSD a given USM can mint, so a second independently maintained exposure cap may not add much protection when both values are intended to represent the same effective capacity.

If retaining both caps is preferred, consider making one value derive from the other or providing a single governance helper that updates both atomically and emits an event showing the paired configuration. The goal is to avoid a state where one stale limit silently blocks liquidations while the other appears correctly configured.

Alto: Acknowledged.

Phaze: Acknowledged. Alto clarifies that the `DUSD` minter ceiling and `AdvancedPermissionsUsm` exposure cap are only equivalent when the price ratio between the underlying asset and `DUSD` is 1:1; in other cases the values may intentionally differ, and the team commits to managing them carefully.

3.2.5 USM seize could handle accrued stable-token fees more explicitly

Severity: Informational.

Context: [AdvancedPermissionsUsm.sol#L216](#).

Description: `AdvancedPermissionsUsm.seize()` is an emergency shutdown function that marks the USM as seized, clears current exposure, clears accrued fees, sets the exposure cap to zero, and transfers the USM's full underlying-asset balance to the stable token treasury.

The function clears `_accruedFees` but does not transfer the stable-token fee balance before doing so. As a result, any stable-token fees already held by the USM remain in the contract after seizure, but they are no longer reflected in `_accruedFees`.

This is not a material security issue because an authorized token rescuer can still recover the remaining stable-token balance after seizure. However, the accounting semantics are less clean: fees that were previously tracked as accrued fees become ordinary unaccounted tokens held by the USM.

Since `seize()` is an emergency-state function, this may be acceptable operationally. Still, handling the fee balance explicitly during seizure would make the shutdown flow easier to reason about and reduce the need for a follow-up rescue action.

Recommendation: Consider moving or distributing accrued stable-token fees before clearing `_accruedFees` in `seize()`. One approach could be to call the existing fee distribution flow before zeroing the seized state, when a fee recipient is configured.

For example:

```
function seize() external notSeized onlyRole(LIQUIDATOR_ROLE) returns (uint256) {
+   distributeFeesToFeeRecipient();
    _isSeized = true;
    _currentExposure = 0;
    _accruedFees = 0;
    _updateExposureCap(0);
    //...
}
```

Alternatively, the function could transfer the accrued stable-token fee balance to the stable token treasury or emit a dedicated event indicating that fee accounting has been intentionally cleared and the remaining tokens should be rescued separately.

Alto: Acknowledged.

Phaze: Acknowledged.

3.2.6 Advanced USM compatibility is not explicitly discoverable

Severity: Informational.

Context: [AdvancedPermissionsUsm.sol#L31](#).

Description: The liquidation periphery depends on advanced USM functionality when sourcing DUSD for liquidations. In particular, it calls the originator-aware advanced quote function before selling the selected USM's underlying asset. Legacy USMs expose the same high-level swap functions, but their quote functions do not include the originator argument and therefore use different function selectors.

Both legacy and advanced USMs inherit OpenZeppelin access control, which exposes ERC165 `supportsInterface()`. However, the advanced USM does not currently advertise support for the advanced permissions USM interface through ERC165. As a result, callers and off-chain integrations need to infer compatibility from deployment context, ABI knowledge, or trial calls rather than from a clear, non-reverting on-chain signal.

This does not create a direct loss-of-funds path by itself. The worst-case outcome is that a legacy USM is selected for the liquidation periphery, the advanced quote call reverts, and the attempted liquidation route fails. The concern is mainly operational: multiple USM generations can make route discovery ambiguous for liquidators, frontends, keepers, and deployment checks, causing avoidable user or operator confusion.

Recommendation: Consider making advanced USM compatibility explicit through ERC165. Since both legacy and advanced USMs already expose `supportsInterface()`, the advanced USM can override it to also report support for `IAdvancedPermissionsUsm`:

```
function supportsInterface(bytes4 interfaceId) public view override returns (bool) {
    return interfaceId == type(IAdvancedPermissionsUsm).interfaceId ||
        ↪ super.supportsInterface(interfaceId);
}
```

Integrations can then safely call the same discovery function on both generations:

```
if (!IERC165(usm).supportsInterface(type(IAdvancedPermissionsUsm).interfaceId)) {
    revert InvalidInput();
}
```

Legacy USMs would return `false`, while advanced USMs would return `true`. This avoids relying on an advanced-only quote call reverting as the compatibility check.

Alto: Fixed in [PR 536](#).

Phaze: [PR 536](#) addresses this by introducing a `USMType` enum for on-chain identification of USM compatibility, as an alternative to the ERC165 override suggested in the finding. Existing `Usm` contracts will be updated to support `USMType` in a separate task.

3.2.7 DUSD is closely linked to frxUSD through the liquidation USM

Severity: Informational.

Context: [AdvancedPermissionsUsm.sol#L31](#).

Description: The advanced permissions frxUSD USM is intended to support liquidations by allowing the liquidation periphery to sell frxUSD into the USM and receive freshly minted DUSD. The planned deployment also makes the buy side permissionless with a 5 bps fee, meaning any user can sell DUSD into the USM to buy frxUSD.

This creates a public redemption route that should keep DUSD closely pegged to frxUSD while the USM has frxUSD liquidity. If DUSD trades below frxUSD by more than the fee and execution costs, users can buy discounted DUSD and redeem it through the USM for frxUSD. As a result, the configuration intentionally ties DUSD's practical redemption value closely to frxUSD through this USM.

The planned deployment sets both the USM exposure cap and the DUSD mint ceiling to `5,000,000,000`, while the USM price strategy values frxUSD and DUSD at a fixed 1:1 ratio. This capacity is effectively unlimited relative to normal liquidation demand. That supports the intended goal of keeping liquidations available during large market moves, but it also means the liquidation route can mint a very large amount of DUSD against frxUSD at par as long as the USM has remaining exposure and DUSD mint ceiling.

This is mostly a configuration and risk-management consideration rather than a code vulnerability. The main scenario to account for is a frxUSD depeg or impairment. If frxUSD trades materially below other stable assets while the USM continues valuing it at 1:1 against DUSD, liquidations can still route through the USM and mint DUSD against the impaired asset at the fixed price. The permissionless 5 bps buy route would also tend to pull DUSD toward frxUSD's market value rather than toward a stronger external dollar reference. This may be acceptable if the protocol explicitly treats frxUSD as reliable backing for emergency liquidation liquidity, but the effective size of the cap makes that assumption important.

Recommendation: Consider using an oracle-aware pricing strategy for the frxUSD USM instead of a fixed 1:1 price. This would allow the USM to account for frxUSD market conditions and avoid continuing to closely link DUSD redemption value and liquidation minting capacity to frxUSD at par if frxUSD materially depegs or becomes impaired.

Alto: Acknowledged.

Phaze: Acknowledged.